Forecasting Stock Market Volatility Using Daily S&P 500 ETF Returns

He Jiang, Patrick Zhu

2026-04-06

```r
library(quantmod)
library(zoo)
library(xts)
library(readr)
library(dplyr)
library(ggplot2)
library(forecast)
library(tseries)
```

Introduction

This project studies stock market volatility using daily S&P 500 ETF returns. Instead of forecasting raw stock prices, we focus on volatility because financial returns often display volatility clustering and are more appropriate for time series modeling.

Data Download and Preparation

We use daily data for the SPDR S&P 500 ETF Trust (SPY) downloaded from Yahoo Finance. Adjusted closing prices are used to compute daily log returns, and volatility is measured as the 21-day rolling standard deviation of returns.

```r
# Download SPY data from Yahoo Finance
getSymbols("SPY", src = "yahoo", from = "2015-01-01")
```

```
## [1] "SPY"
```

```r
# Extract adjusted close price
spy_adj <- Ad(SPY)

# Compute daily log returns
spy_ret <- diff(log(spy_adj))

# Compute 21-day rolling volatility
spy_vol_21 <- rollapply(
  spy_ret,
  width = 21,
  FUN = sd,
  align = "right",
  fill = NA
)
```

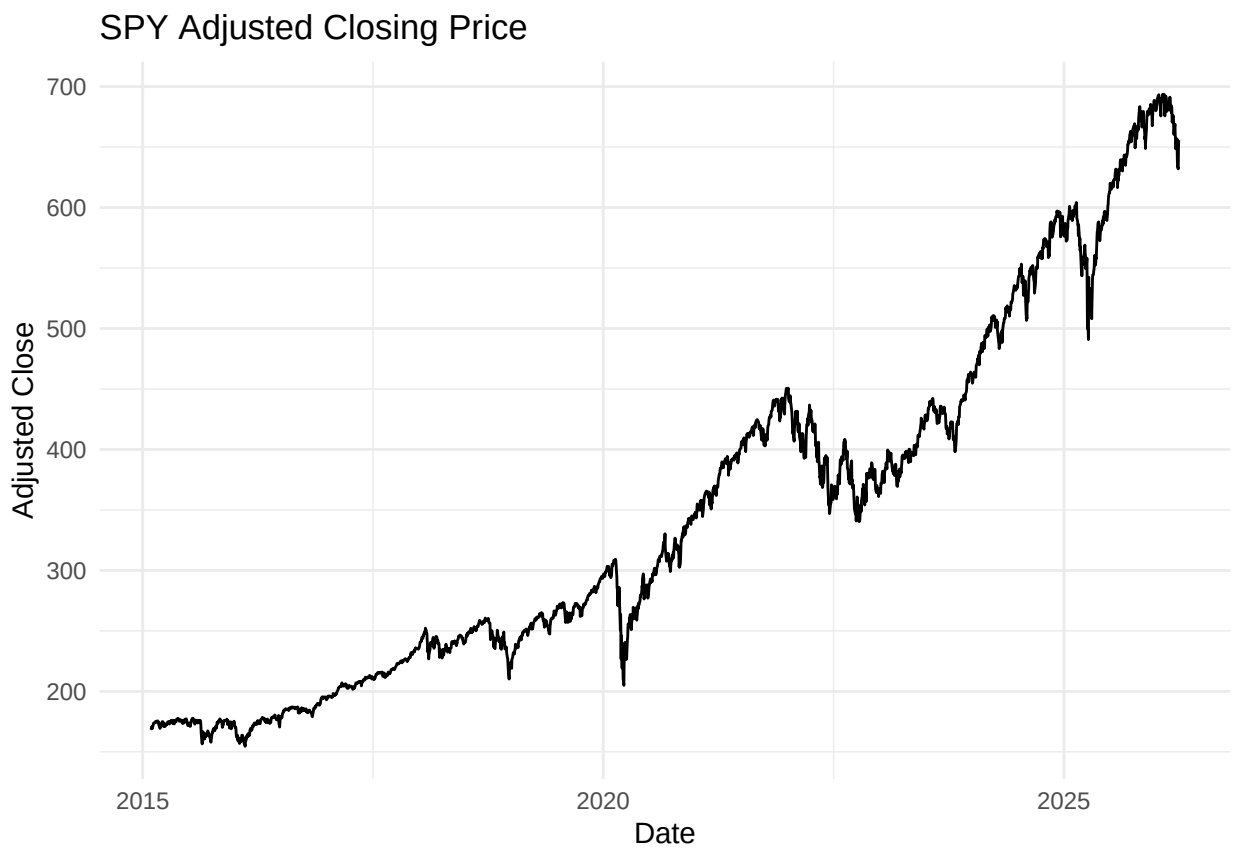
```r
# Build data frame
data <- data.frame(
  date = as.Date(index(spy_vol_21)),
  adj_close = as.numeric(spy_adj[index(spy_vol_21)]),
  return = as.numeric(spy_ret[index(spy_vol_21)]),
  volatility_21 = as.numeric(spy_vol_21)
)

data <- na.omit(data)

head(data)
```

```
##          date adj_close       return volatility_21
## 22 2015-02-03  169.6364  0.014357489   0.011266124
## 23 2015-02-04  168.9904 -0.003815211   0.010523740
## 24 2015-02-05  170.6965  0.010044771   0.010456509
## 25 2015-02-06  170.2244 -0.002769242   0.010185616
## 26 2015-02-09  169.4625 -0.004486184   0.009475981
## 27 2015-02-10  171.2678  0.010597185   0.009586142
```

Exploratory Data Analysis

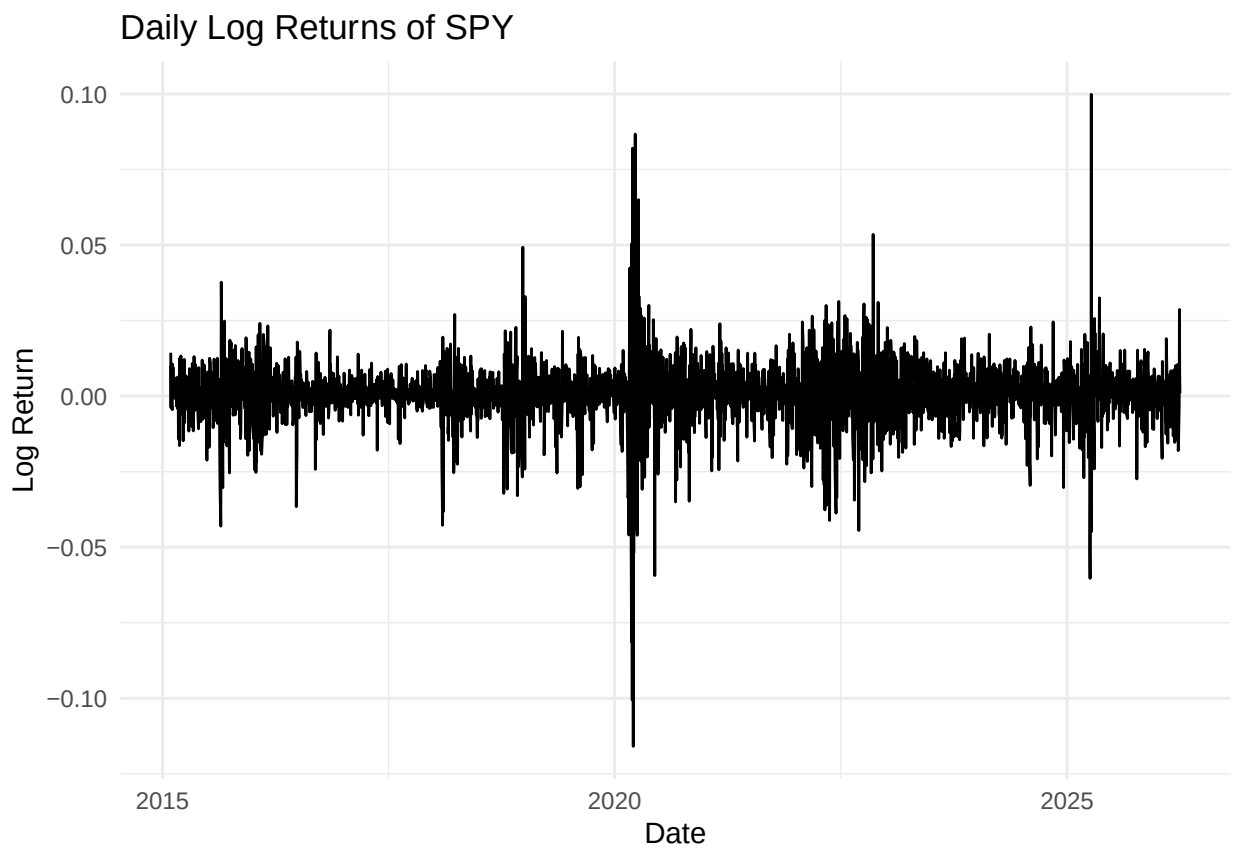
SPY Adjusted Close

```r
ggplot(data, aes(x = date, y = adj_close)) +
  geom_line() +
  labs(title = "SPY Adjusted Closing Price", x = "Date", y = "Adjusted Close") +
  theme_minimal()
```



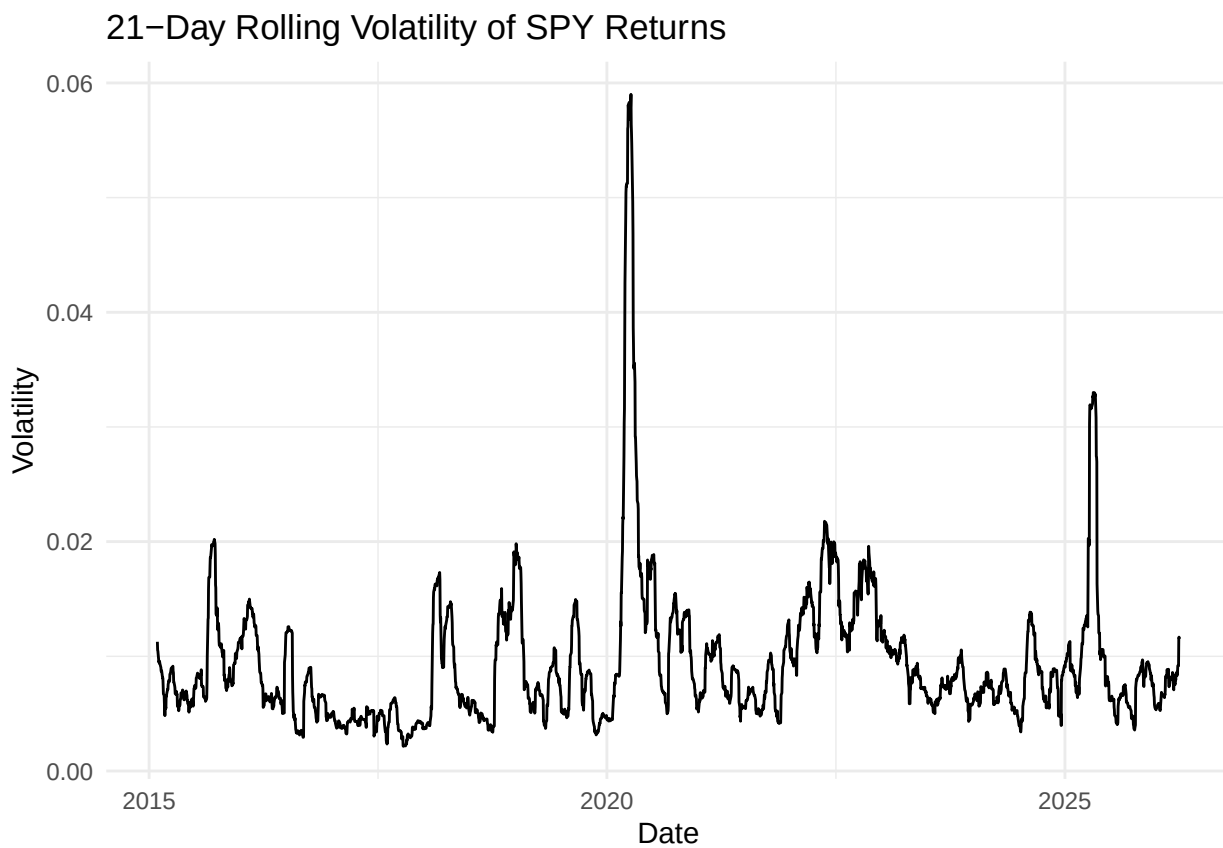
Daily Returns

```
ggplot(data, aes(x = date, y = return)) +
  geom_line() +
  labs(title = "Daily Log Returns of SPY", x = "Date", y = "Log Return") +
  theme_minimal()
```



21-Day Rolling Volatility

```
ggplot(data, aes(x = date, y = volatility_21)) +
  geom_line() +
  labs(title = "21-Day Rolling Volatility of SPY Returns", x = "Date", y = "Volatility") +
  theme_minimal()
```



Preliminary Time Series Analysis

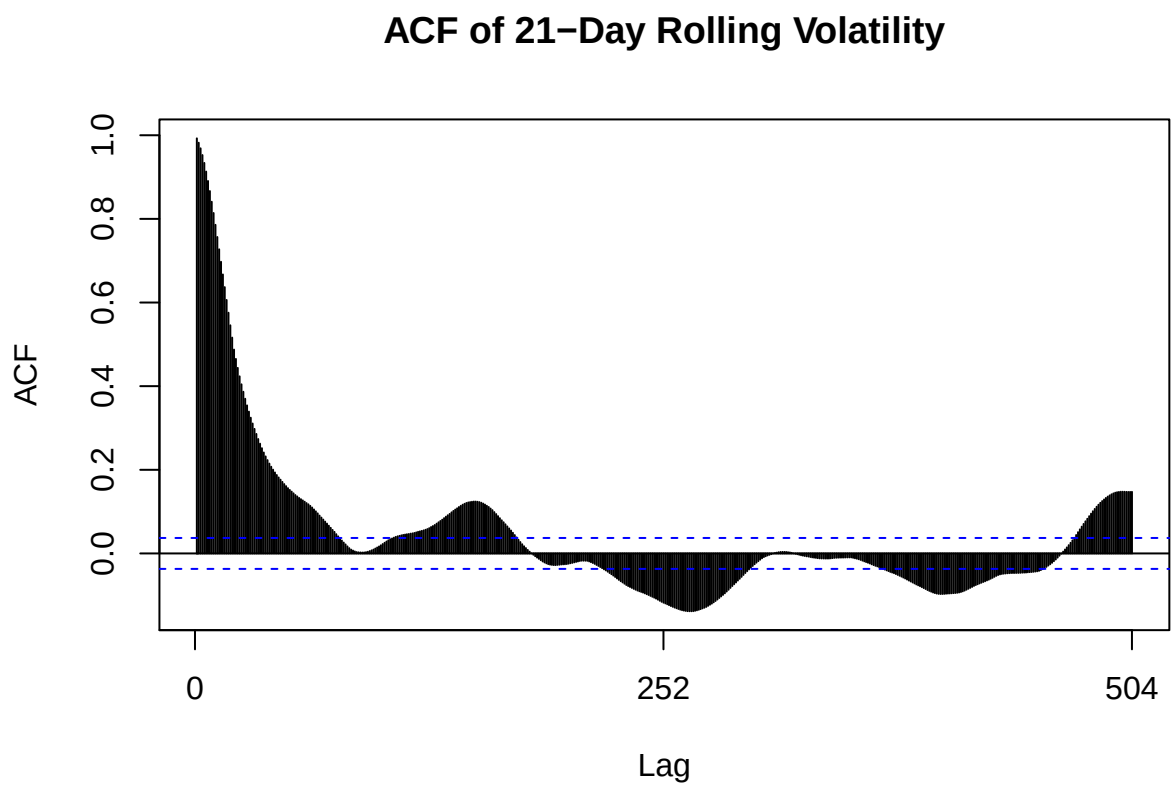
Summary Statistics

```
summary(data)
```

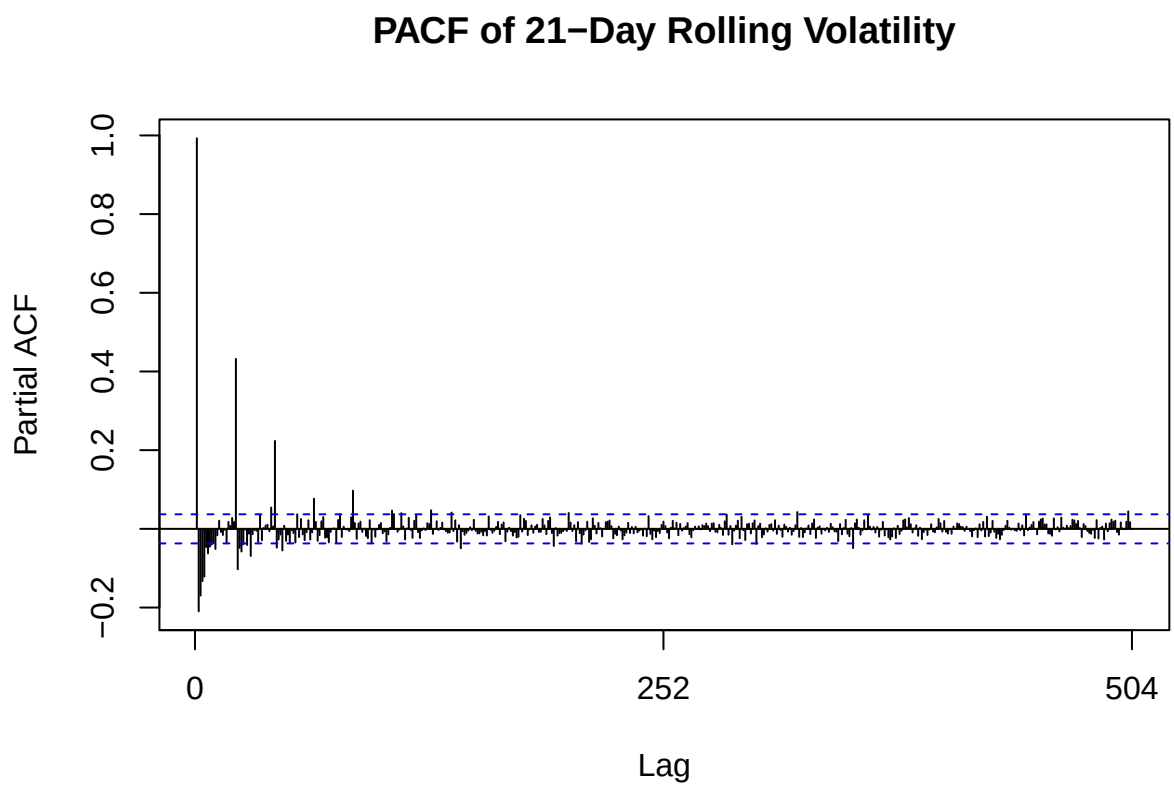
```
##       date              adj_close         return           volatility_21
##  Min.   :2015-02-03   Min.   :154.6   Min.   :-0.1158864   Min.   :0.002152
##  1st Qu.:2017-11-13   1st Qu.:224.2   1st Qu.:-0.0036828   1st Qu.:0.005781
##  Median :2020-08-29   Median :309.3   Median : 0.0006117   Median :0.007775
##  Mean   :2020-08-30   Mean   :345.9   Mean   : 0.0004867   Mean   :0.009365
##  3rd Qu.:2023-06-14   3rd Qu.:428.0   3rd Qu.: 0.0058973   3rd Qu.:0.011423
##  Max.   :2026-04-02   Max.   :693.6   Max.   : 0.0998628   Max.   :0.059005
```

ACF and PACF of Volatility

```
vol_ts <- ts(data$volatility_21, frequency = 252)
Acf(vol_ts, main = "ACF of 21-Day Rolling Volatility")
```




```
Pacf(vol_ts, main = "PACF of 21-Day Rolling Volatility")
```



Next Steps

The next stage of the project will split the data into training and test sets and compare forecasting performance across multiple models, such as ARIMA and GARCH.

Train/Test Split

We split the volatility series into a training set and a test set. The training set is used to fit the models, and the test set is used to evaluate out-of-sample forecasting performance.

```
n <- nrow(data)
train_size <- floor(0.8 * n)

train_data <- data[1:train_size, ]
test_data  <- data[(train_size + 1):n, ]

train_vol <- train_data$volatility_21
test_vol  <- test_data$volatility_21

length(train_vol)
```

```
## [1] 2246
```

```
length(test_vol)
```

```
## [1] 562
```

```r
tail(train_data)
```

```
##            date adj_close        return volatility_21
## 2262 2023-12-27  463.7478  0.0018064371   0.005743126
## 2263 2023-12-28  463.9230  0.0003776574   0.005752433
## 2264 2023-12-29  462.5800 -0.0028991132   0.005831408
## 2265 2024-01-02  459.9912 -0.0056121033   0.006066157
## 2266 2024-01-03  456.2346 -0.0082001898   0.006369304
## 2267 2024-01-04  454.7650 -0.0032262915   0.006281051
```

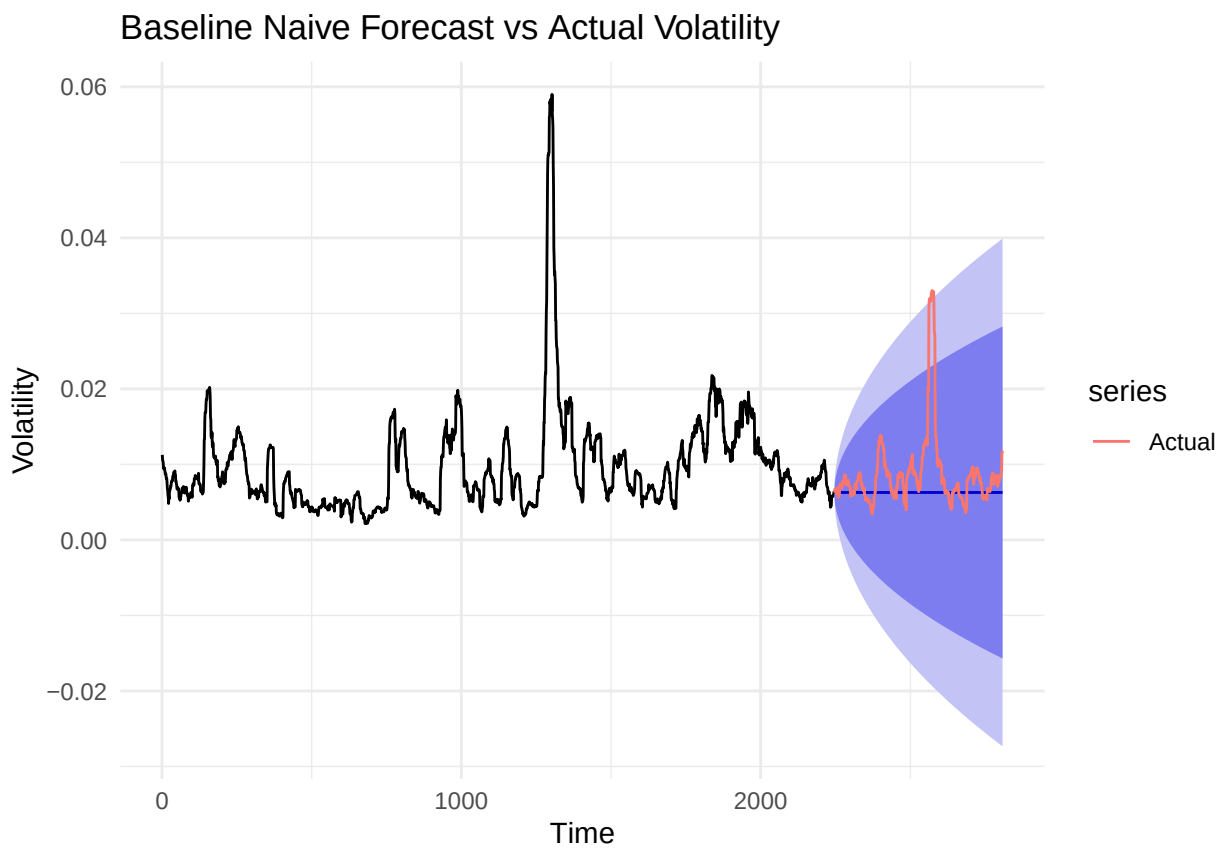
```r
head(test_data)
```

```
##            date adj_close        return volatility_21
## 2268 2024-01-05  455.3879  0.0013687015   0.006271876
## 2269 2024-01-08  461.8890  0.0141750160   0.006730755
## 2270 2024-01-09  461.1883 -0.0015182797   0.006660433
## 2271 2024-01-10  463.7965  0.0056396241   0.006692181
## 2272 2024-01-11  463.5921 -0.0004408880   0.006693235
## 2273 2024-01-12  463.9132  0.0006924736   0.006662379
```

Baseline Forecast

As a simple benchmark, we first use a naive forecasting approach. This baseline helps us evaluate whether more advanced models such as ARIMA and GARCH actually improve forecasting accuracy.

```r
train_ts <- ts(train_vol, start = 1, frequency = 1)
test_ts  <- ts(test_vol,  start = length(train_ts) + 1, frequency = 1)

baseline_fit <- naive(train_ts, h = length(test_ts))

autoplot(baseline_fit) +
  autolayer(test_ts, series = "Actual") +
  labs(
    title = "Baseline Naive Forecast vs Actual Volatility",
    x = "Time",
    y = "Volatility"
  ) +
  theme_minimal()
```




```r
accuracy(baseline_fit, test_ts)
```

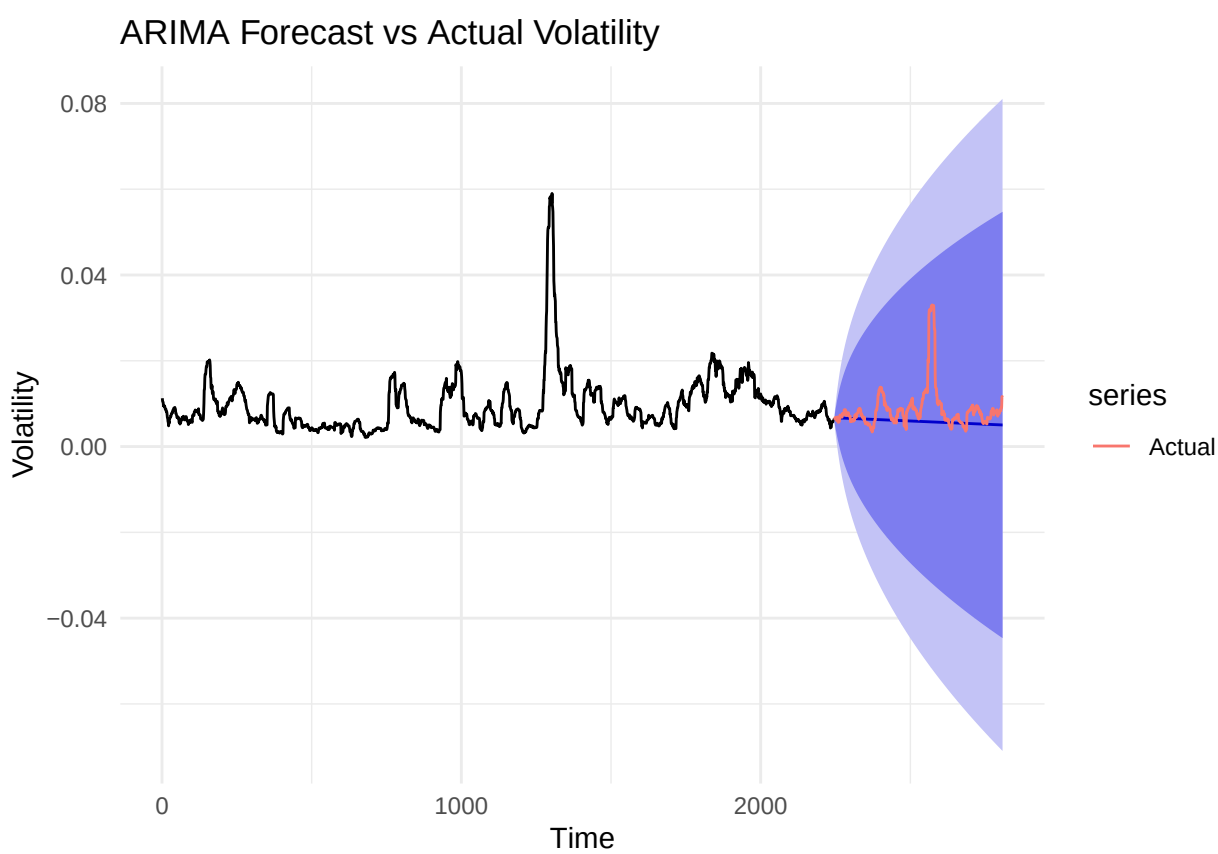
```
##                          ME         RMSE         MAE        MPE      MAPE
## Training set -2.220522e-06 0.0007234041 0.000354993 -0.2804592  3.867872
## Test set      2.454113e-03 0.0056461971 0.002951350 15.2652942 26.035758
##                  MASE      ACF1 Theil's U
## Training set 1.000000 0.2228746        NA
## Test set     8.313826 0.9849643  3.701728
```

```r
arima_fit <- auto.arima(train_ts, seasonal = FALSE)
summary(arima_fit)
```

```
## Series: train_ts
## ARIMA(3,1,2) with drift
##
## Coefficients:
##           ar1     ar2     ar3     ma1      ma2  drift
##       -0.1350  0.8373  0.0355  0.2708  -0.6339      0
## s.e.   0.0533  0.0275  0.0295  0.0494   0.0425      0
##
## sigma^2 = 4.598e-07:  log likelihood = 13197.28
## AIC=-26380.56   AICc=-26380.51   BIC=-26340.55
##
## Training set error measures:
##                        ME         RMSE          MAE         MPE     MAPE
## Training set 5.214112e-07 0.0006770593 0.0003653542 0.003349421 4.051745
##                  MASE         ACF1
```

```
## Training set 1.029187 0.0005024915
```

```r
arima_fc <- forecast(arima_fit, h = length(test_ts))

autoplot(arima_fc) +
  autolayer(test_ts, series = "Actual") +
  labs(
    title = "ARIMA Forecast vs Actual Volatility",
    x = "Time",
    y = "Volatility"
  ) +
  theme_minimal()
```




```r
accuracy(arima_fc, test_ts)
```

```
##                        ME         RMSE          MAE          MPE      MAPE
## Training set 5.214112e-07 0.0006770593 0.0003653542  0.003349421  4.051745
## Test set     2.860616e-03 0.0058775297 0.0032297442 20.645802426 28.729916
##                  MASE         ACF1 Theil's U
## Training set 1.029187 0.0005024915        NA
## Test set     9.098049 0.9849879256  3.972679
```

```r
baseline_acc <- accuracy(baseline_fit, test_ts)
arima_acc    <- accuracy(arima_fc, test_ts)

comparison <- data.frame(
  Model = c("Naive Baseline", "ARIMA"),
  RMSE  = c(baseline_acc[2, "RMSE"], arima_acc[2, "RMSE"]),
```

```r
  MAE   = c(baseline_acc[2, "MAE"],  arima_acc[2, "MAE"]),
  MAPE  = c(baseline_acc[2, "MAPE"], arima_acc[2, "MAPE"])
)

comparison
```

```
##            Model        RMSE         MAE     MAPE
## 1 Naive Baseline 0.005646197 0.002951350 26.03576
## 2          ARIMA 0.005877530 0.003229744 28.72992
```

GARCH Model

```r
library(tseries)

# Use return series for GARCH
train_ret <- train_data$return
test_ret  <- test_data$return

# Fit GARCH(1,1)
garch_fit <- garch(train_ret, order = c(1, 1))
```

```
##
##  ***** ESTIMATION WITH ANALYTICAL GRADIENT *****
##
##
##      I     INITIAL X(I)        D(I)
##
##      1     1.177691e-04     1.000e+00
##      2     5.000000e-02     1.000e+00
##      3     5.000000e-02     1.000e+00
##
##     IT   NF      F         RELDF    PRELDF    RELDX   STPPAR   D*STEP   NPRELDF
##      0    1 -9.056e+03
##      1    7 -9.069e+03  1.48e-03  1.75e-03  1.0e-04  1.6e+11  1.0e-05  1.39e+08
##      2    8 -9.076e+03  7.66e-04  8.73e-04  1.0e-04  3.0e+00  1.0e-05  1.52e+02
##      3    9 -9.077e+03  4.40e-05  5.93e-05  9.2e-05  2.0e+00  1.0e-05  1.42e+02
##      4   10 -9.077e+03  5.43e-06  5.65e-06  9.9e-05  2.0e+00  1.0e-05  1.41e+02
##      5   18 -9.165e+03  9.67e-03  1.62e-02  4.8e-01  2.0e+00  9.3e-02  1.39e+02
##      6   21 -9.292e+03  1.36e-02  1.61e-02  7.2e-01  2.0e+00  3.2e-01  1.16e+01
##      7   22 -9.301e+03  9.90e-04  2.91e-02  2.9e-01  2.0e+00  3.2e-01  7.54e+00
##      8   27 -9.482e+03  1.90e-02  1.53e-02  1.2e-05  2.6e+00  1.6e-05  3.62e-01
##      9   30 -9.483e+03  1.62e-04  2.70e-04  6.4e-07  6.6e+00  8.7e-07  2.92e+00
##     10   31 -9.483e+03  4.09e-07  5.14e-07  6.3e-07  2.0e+00  8.7e-07  7.90e-01
##     11   40 -9.491e+03  8.27e-04  1.89e-03  3.9e-02  2.0e+00  5.6e-02  7.85e-01
##     12   41 -9.492e+03  1.04e-04  1.77e-04  5.2e-03  0.0e+00  9.6e-03  1.77e-04
##     13   42 -9.493e+03  8.60e-05  5.08e-05  4.3e-03  0.0e+00  8.4e-03  5.08e-05
##     14   43 -9.494e+03  1.36e-04  1.33e-04  1.3e-02  4.8e-01  2.7e-02  1.55e-04
##     15   44 -9.495e+03  7.30e-05  6.36e-05  1.4e-02  1.4e-01  2.7e-02  6.42e-05
##     16   45 -9.495e+03  2.33e-05  2.39e-05  5.8e-03  0.0e+00  1.3e-02  2.39e-05
##     17   46 -9.495e+03  4.19e-07  6.18e-07  1.1e-03  0.0e+00  1.9e-03  6.18e-07
##     18   47 -9.495e+03  8.30e-08  1.07e-07  1.6e-04  0.0e+00  3.3e-04  1.07e-07
##     19   63 -9.495e+03 -8.62e-15  3.04e-15  4.5e-15  5.7e+04  7.1e-15  1.20e-09
##
##  ***** FALSE CONVERGENCE *****
```

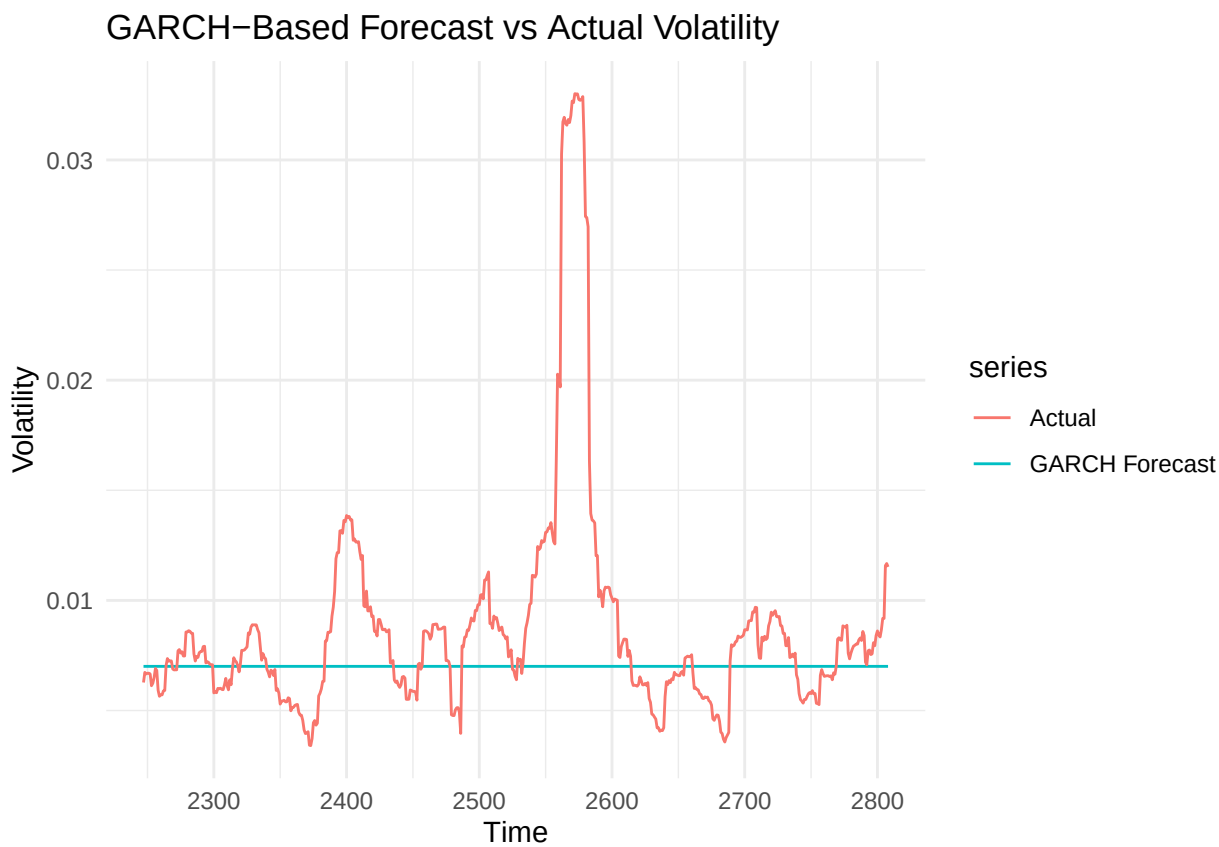
```
##
##  FUNCTION    -9.495040e+03   RELDX        4.542e-15
##  FUNC. EVALS      63         GRAD. EVALS      19
##  PRELDF       3.041e-15      NPRELDF      1.203e-09
##
##      I      FINAL X(I)        D(I)          G(I)
##
##      1    3.758571e-06     1.000e+00     4.043e+03
##      2    1.859264e-01     1.000e+00     3.614e-01
##      3    7.861829e-01     1.000e+00     2.513e-01
```

```r
# Show model summary
summary(garch_fit)
```

```
##
## Call:
## garch(x = train_ret, order = c(1, 1))
##
## Model:
## GARCH(1,1)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -5.94964 -0.45648  0.07408  0.68656  3.81694
##
## Coefficient(s):
##     Estimate  Std. Error  t value Pr(>|t|)
## a0 3.759e-06   4.084e-07    9.202   <2e-16 ***
## a1 1.859e-01   1.538e-02   12.086   <2e-16 ***
## b1 7.862e-01   1.598e-02   49.194   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Diagnostic Tests:
##  Jarque Bera Test
##
## data:  Residuals
## X-squared = 678.19, df = 2, p-value < 2.2e-16
##
##
##  Box-Ljung test
##
## data:  Squared.Residuals
## X-squared = 0.31793, df = 1, p-value = 0.5729
```

```r
# Extract fitted conditional volatility from training period
train_garch_vol <- fitted(garch_fit)[, 1]

# Use the last fitted conditional volatility as a simple forecast benchmark
last_garch_vol <- tail(train_garch_vol, 1)
garch_forecast <- rep(last_garch_vol, length(test_vol))

# Convert forecast to time series for plotting
garch_fc_ts <- ts(garch_forecast, start = length(train_ts) + 1, frequency = 1)
```

```r
# Plot GARCH forecast against actual test volatility
autoplot(garch_fc_ts, series = "GARCH Forecast") +
  autolayer(test_ts, series = "Actual") +
  labs(
    title = "GARCH-Based Forecast vs Actual Volatility",
    x = "Time",
    y = "Volatility"
  ) +
  theme_minimal()
```




```r
# Accuracy
garch_acc <- accuracy(garch_fc_ts, test_ts)
garch_acc
```

```
##                    ME        RMSE         MAE      MPE     MAPE      ACF1
## Test set 0.001730412 0.005371331 0.002691577 5.502185 24.84302 0.9849643
##          Theil's U
## Test set  3.666522
```

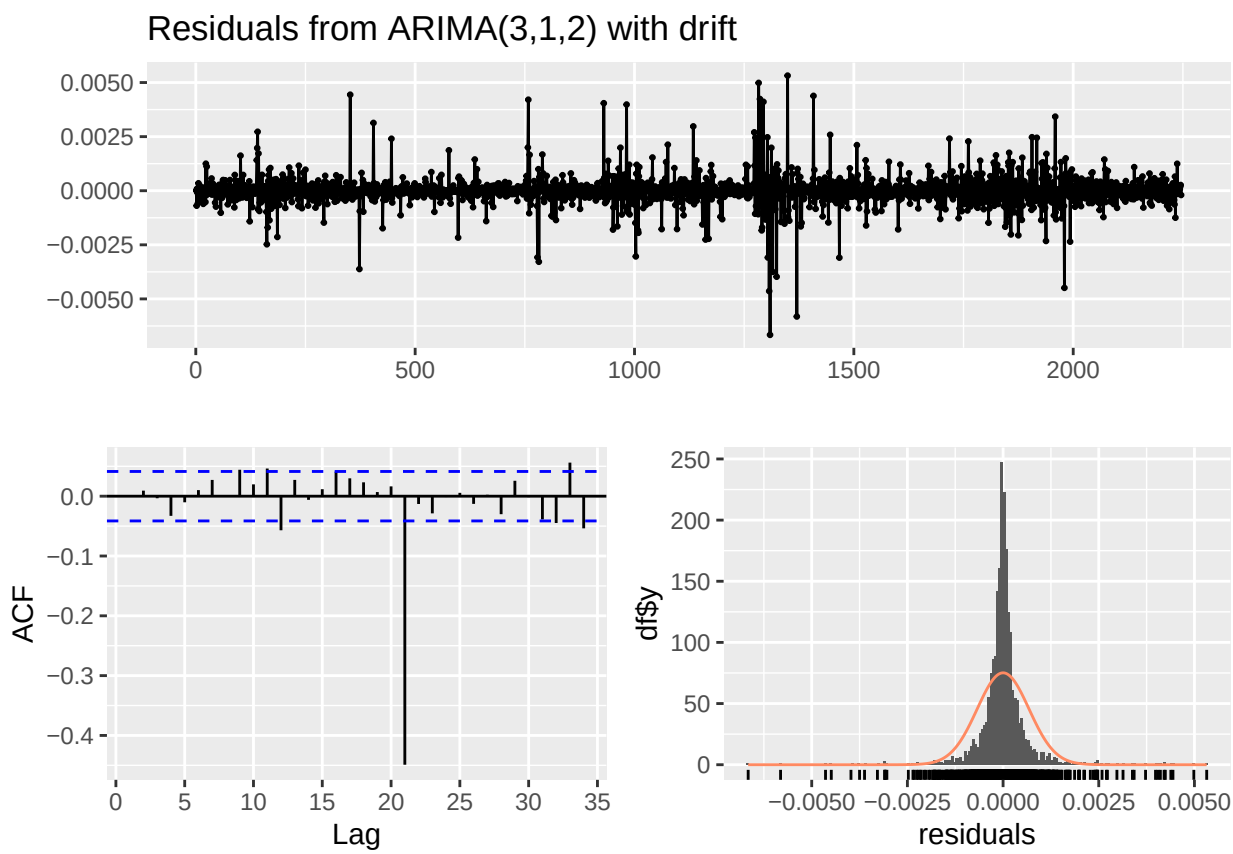
```r
baseline_acc <- accuracy(baseline_fit, test_ts)
arima_acc    <- accuracy(arima_fc, test_ts)
garch_acc    <- accuracy(garch_fc_ts, test_ts)

comparison <- data.frame(
  Model = c("Naive Baseline", "ARIMA", "GARCH-based"),
  RMSE  = c(baseline_acc[2, "RMSE"], arima_acc[2, "RMSE"], garch_acc[1, "RMSE"]),
  MAE   = c(baseline_acc[2, "MAE"],  arima_acc[2, "MAE"],  garch_acc[1, "MAE"]),
```

```
  MAPE  = c(baseline_acc[2, "MAPE"], arima_acc[2, "MAPE"], garch_acc[1, "MAPE"])
)

comparison
```

```
##            Model        RMSE         MAE     MAPE
## 1 Naive Baseline 0.005646197 0.002951350 26.03576
## 2          ARIMA 0.005877530 0.003229744 28.72992
## 3    GARCH-based 0.005371331 0.002691577 24.84302
```

ARIMA Residual Diagnostics

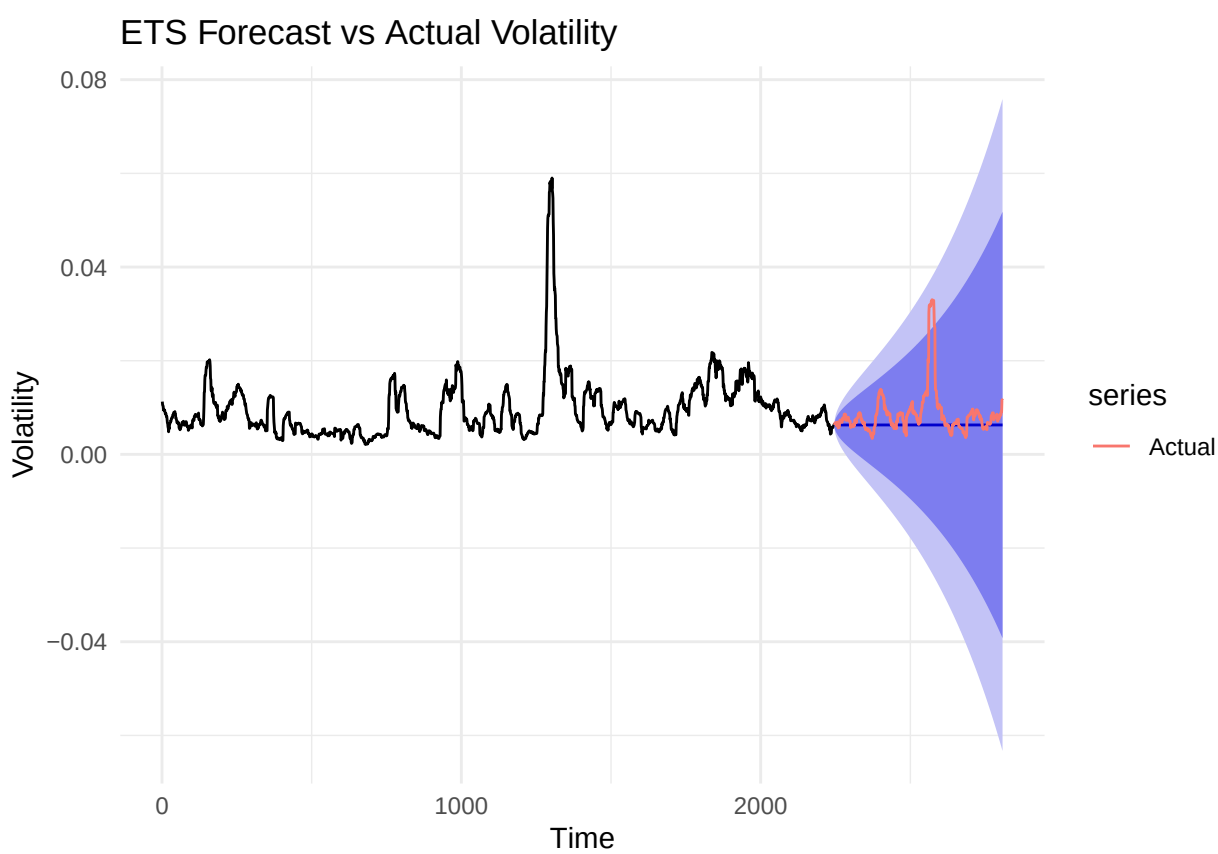
```
checkresiduals(arima_fit)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(3,1,2) with drift
## Q* = 10.086, df = 5, p-value = 0.07283
##
## Model df: 5.   Total lags used: 10
```

ETS Model

```r
ets_fit <- ets(train_ts)
ets_fc  <- forecast(ets_fit, h = length(test_ts))

autoplot(ets_fc) +
  autolayer(test_ts, series = "Actual") +
  labs(
    title = "ETS Forecast vs Actual Volatility",
    x = "Time",
    y = "Volatility"
  ) +
  theme_minimal()
```

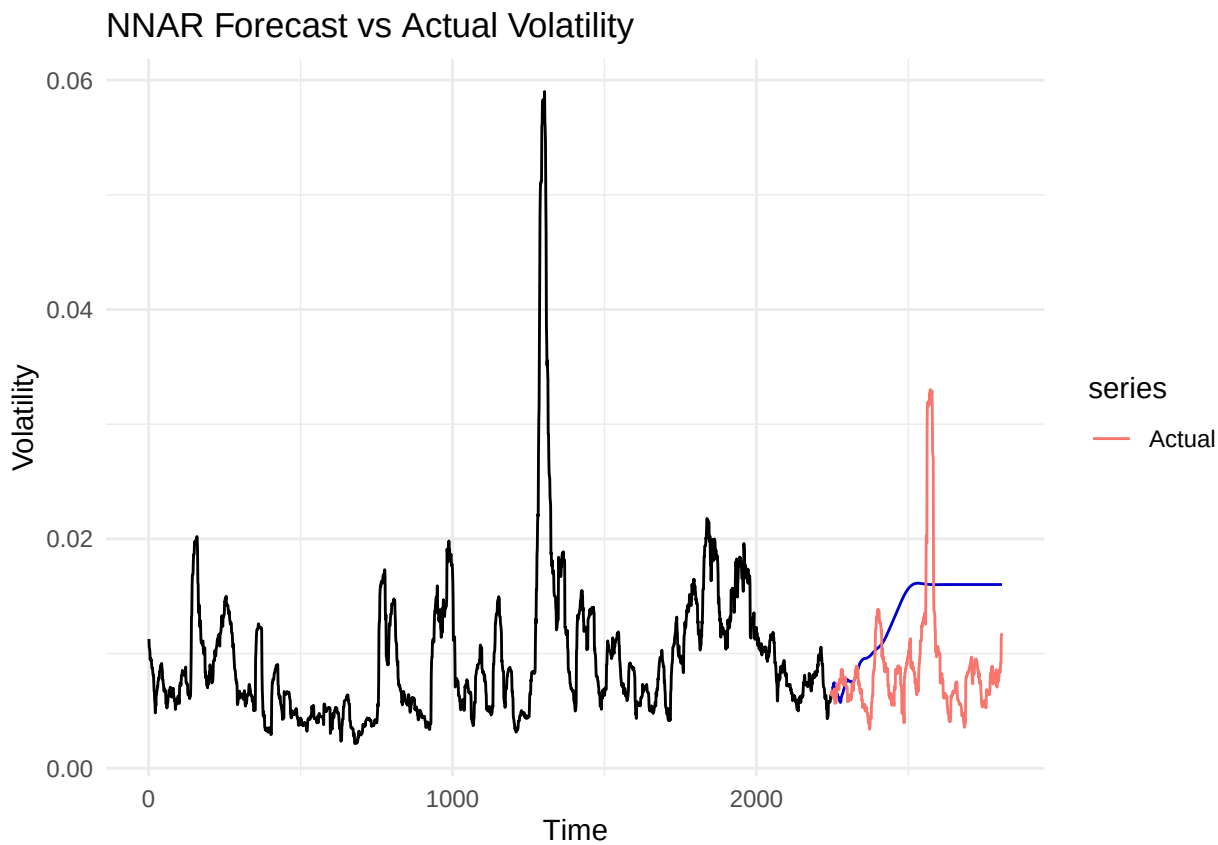



```r
ets_acc <- accuracy(ets_fc, test_ts)
ets_acc
```

```
##                         ME         RMSE          MAE       MPE      MAPE
## Training set -2.193150e-06 0.0007232603 0.0003548679 -0.280132  3.866445
## Test set      2.454104e-03 0.0056461933 0.0029513462 15.265175 26.035727
##                   MASE      ACF1 Theil's U
## Training set 0.9996476 0.2229516        NA
## Test set     8.3138143 0.9849643  3.701725
```

```r
nn_fit <- nnetar(train_ts)
nn_fc  <- forecast(nn_fit, h = length(test_ts))

autoplot(nn_fc) +
  autolayer(test_ts, series = "Actual") +
```

```r
  labs(
    title = "NNAR Forecast vs Actual Volatility",
    x = "Time",
    y = "Volatility"
  ) +
  theme_minimal()
```

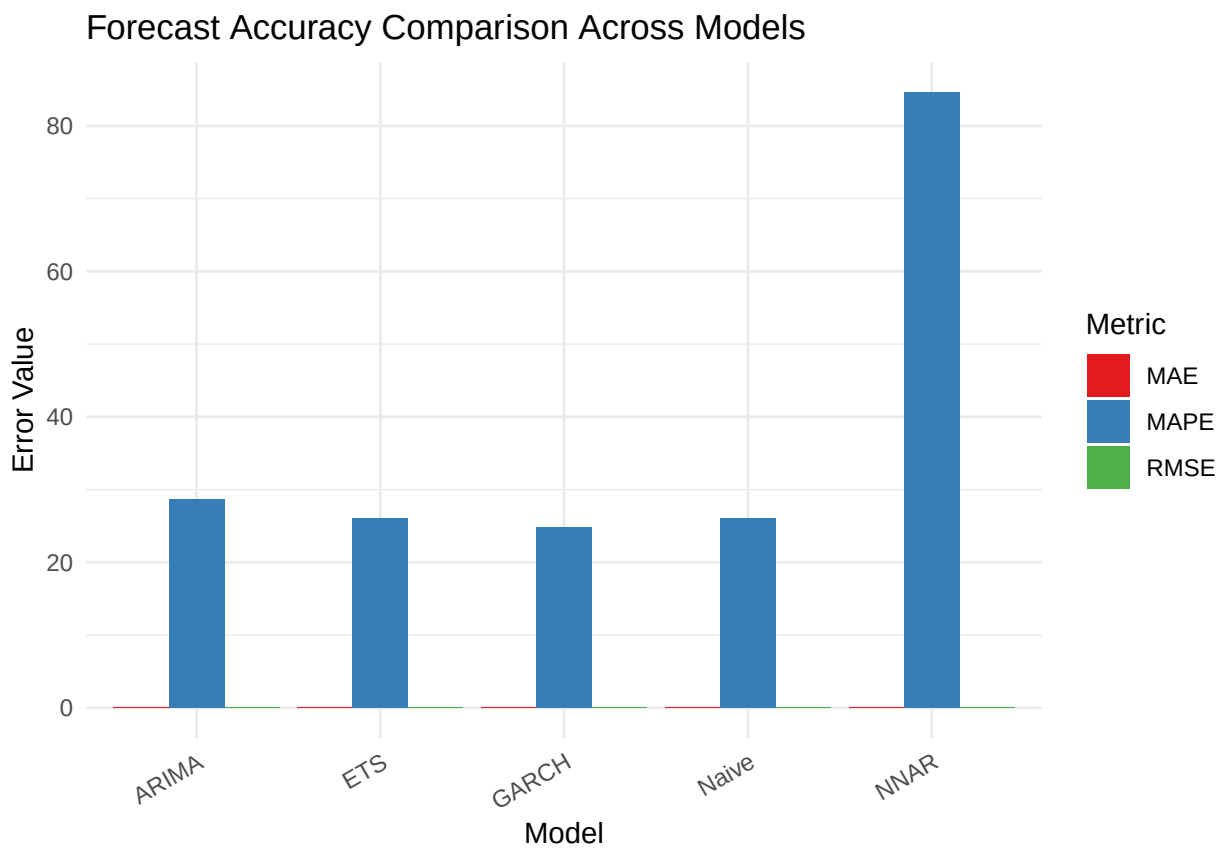



```r
nn_acc <- accuracy(nn_fc, test_ts)
nn_acc
```

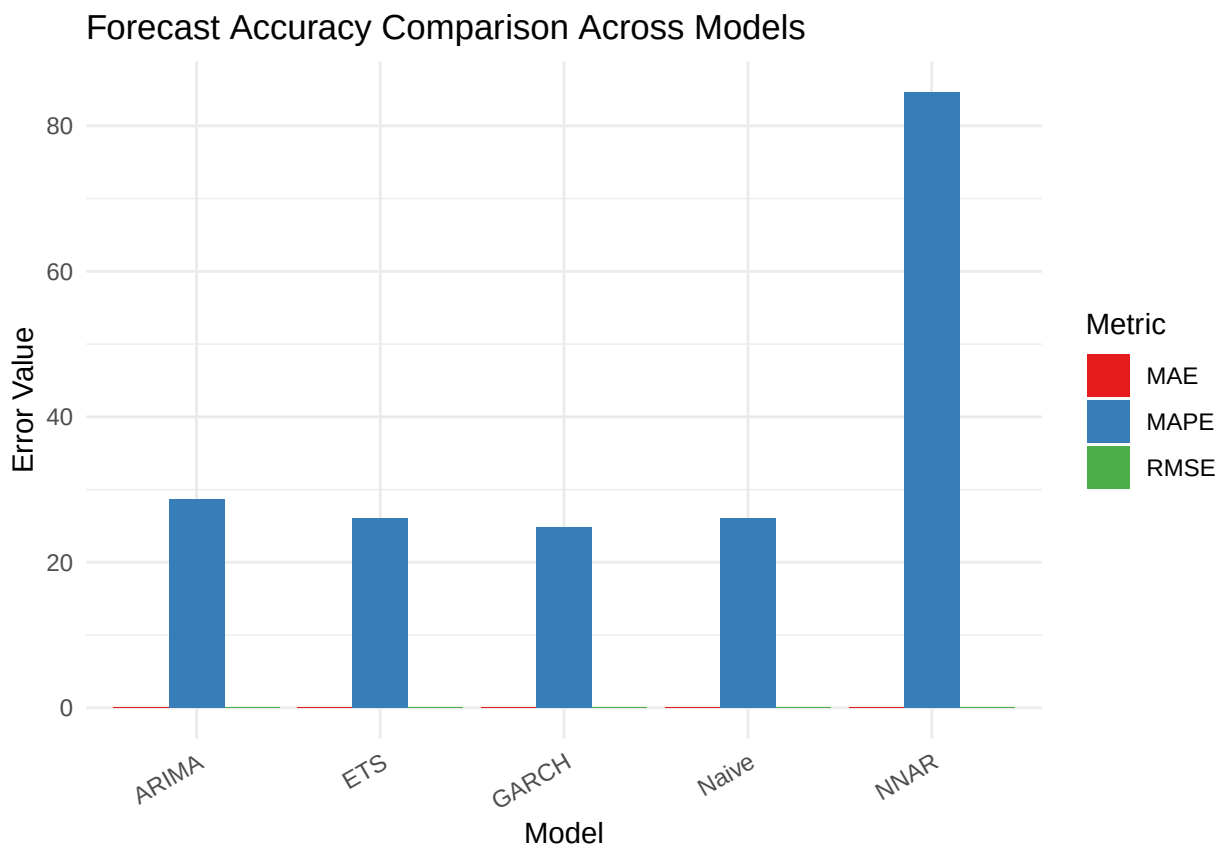
```
##                        ME         RMSE          MAE        MPE      MAPE
## Training set -0.0000013659 0.0004475898 0.0002652949  -0.391488  3.298464
## Test set     -0.0046101886 0.0072258625 0.0061321676 -77.495396 84.583797
##                    MASE         ACF1 Theil's U
## Training set  0.7473243 -0.002831935        NA
## Test set     17.2740502  0.987276310   12.1792
```

```r
results_all <- data.frame(
  Model = c("Naive", "ARIMA", "GARCH", "ETS", "NNAR"),
  RMSE  = c(
    baseline_acc[2, "RMSE"],
    arima_acc[2, "RMSE"],
    garch_acc[1, "RMSE"],
    ets_acc[2, "RMSE"],
    nn_acc[2, "RMSE"]
  ),
  MAE   = c(
```

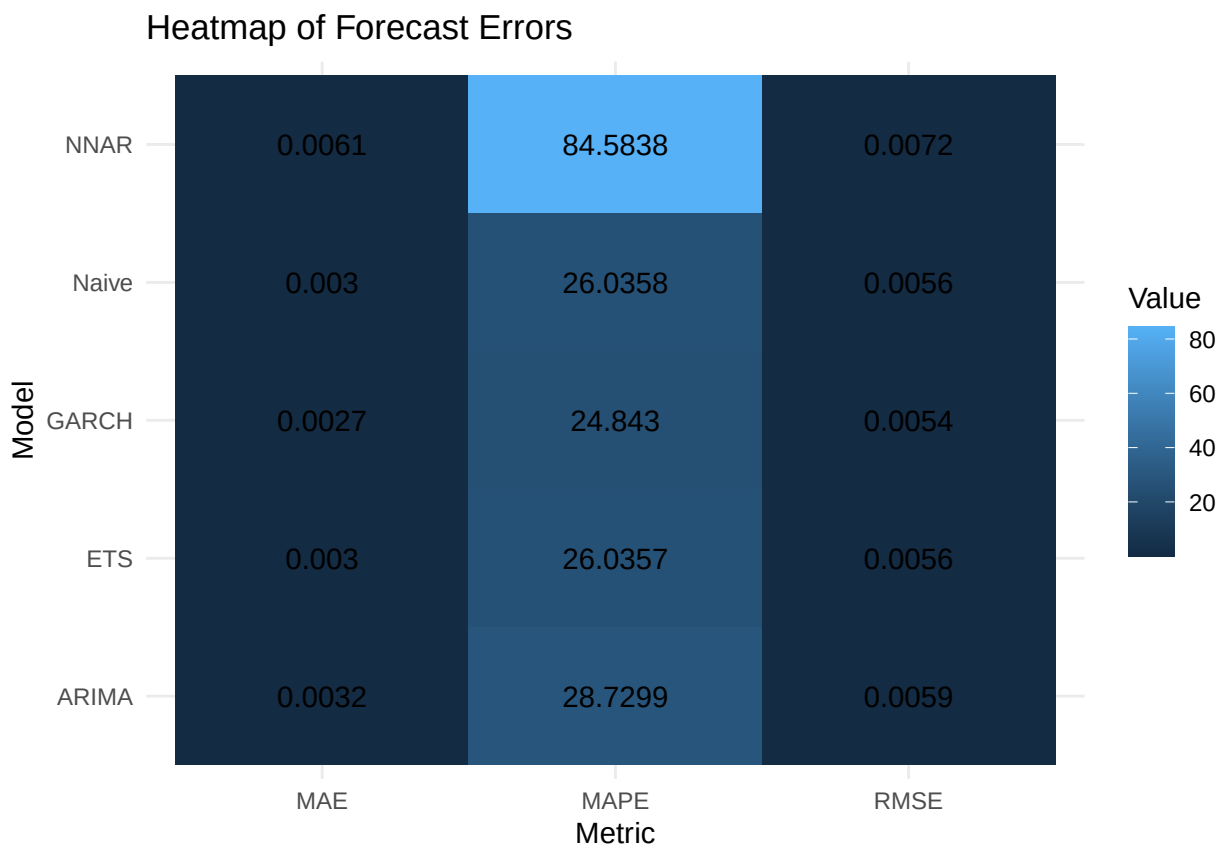
```r
    baseline_acc[2, "MAE"],
    arima_acc[2, "MAE"],
    garch_acc[1, "MAE"],
    ets_acc[2, "MAE"],
    nn_acc[2, "MAE"]
  ),
  MAPE  = c(
    baseline_acc[2, "MAPE"],
    arima_acc[2, "MAPE"],
    garch_acc[1, "MAPE"],
    ets_acc[2, "MAPE"],
    nn_acc[2, "MAPE"]
  )
)

results_long <- data.frame(
  Model = rep(results_all$Model, 3),
  Metric = rep(c("RMSE", "MAE", "MAPE"), each = nrow(results_all)),
  Value = c(results_all$RMSE, results_all$MAE, results_all$MAPE)
)

ggplot(results_long, aes(x = Model, y = Value, fill = Metric)) +
  geom_col(position = "dodge") +
  labs(
    title = "Forecast Accuracy Comparison Across Models",
    x = "Model",
    y = "Error Value"
  ) +
  scale_fill_brewer(palette = "Set1") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 30, hjust = 1))
```



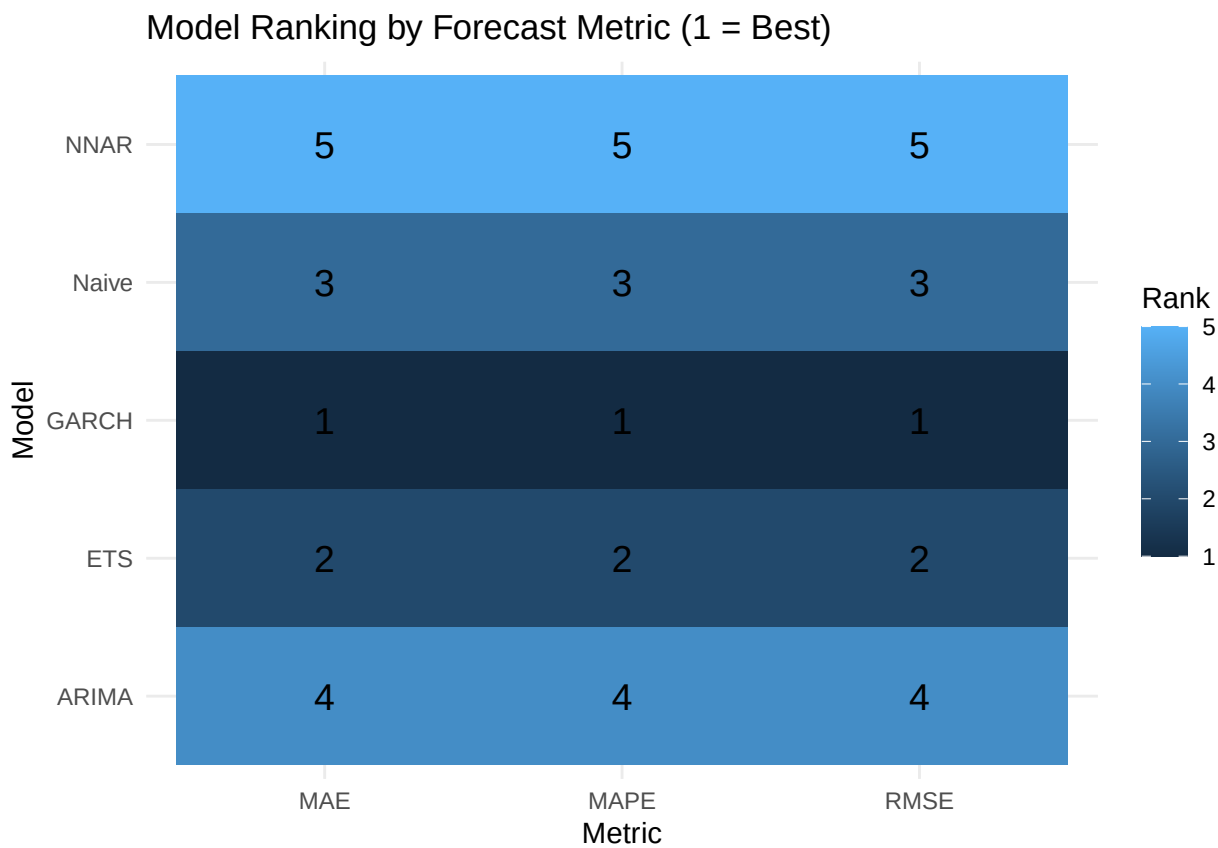

```r
ggplot(results_long, aes(x = Model, y = Value, fill = Metric)) +
  geom_col(position = "dodge") +
  labs(
    title = "Forecast Accuracy Comparison Across Models",
    x = "Model",
    y = "Error Value"
  ) +
  scale_fill_brewer(palette = "Set1") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 30, hjust = 1))
```

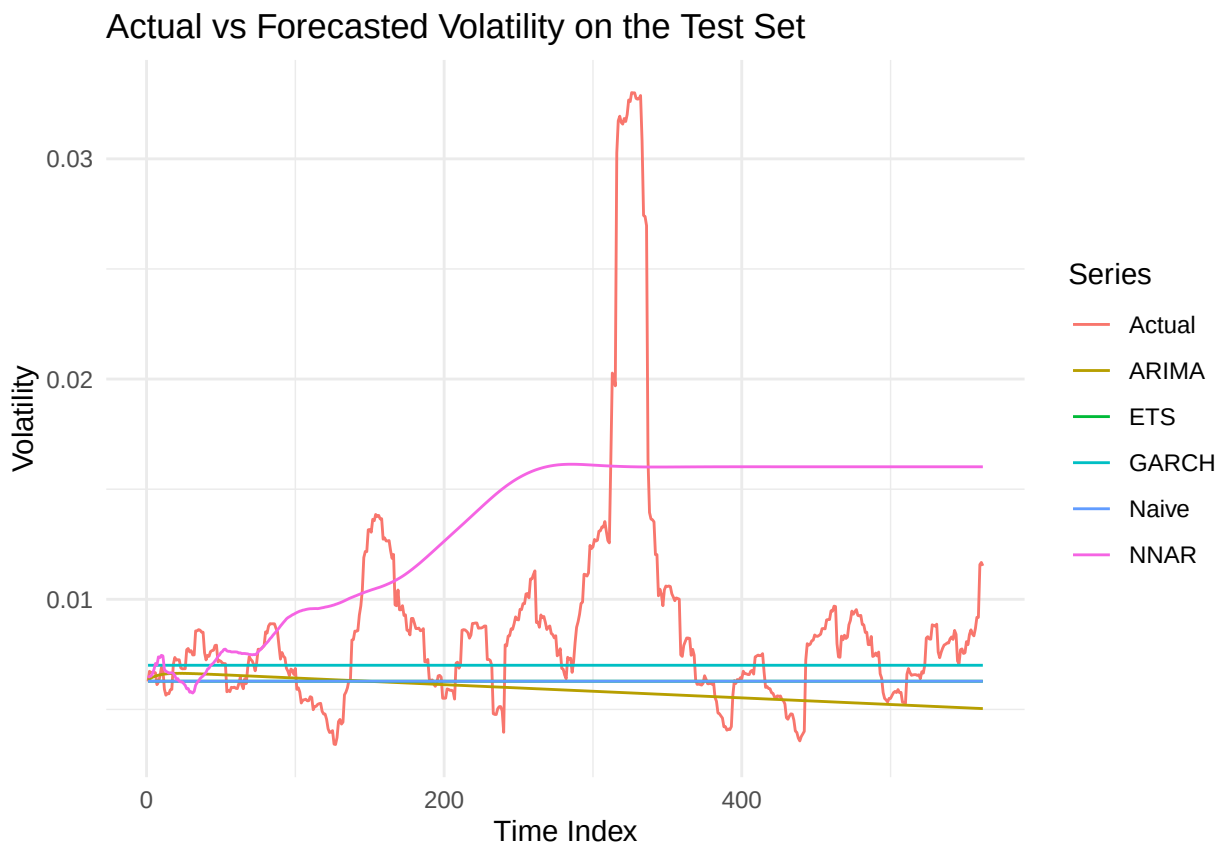



```r
ggplot(results_long, aes(x = Metric, y = Model, fill = Value)) +
  geom_tile() +
  geom_text(aes(label = round(Value, 4)), size = 4) +
  labs(
    title = "Heatmap of Forecast Errors",
    x = "Metric",
    y = "Model"
  ) +
  theme_minimal()
```




```r
results_rank <- data.frame(
  Model = results_all$Model,
  RMSE_rank = rank(results_all$RMSE),
  MAE_rank  = rank(results_all$MAE),
  MAPE_rank = rank(results_all$MAPE)
)


ranking_long <- data.frame(
  Model = rep(results_rank$Model, 3),
  Metric = rep(c("RMSE", "MAE", "MAPE"), each = nrow(results_rank)),
  Rank = c(results_rank$RMSE_rank,
           results_rank$MAE_rank,
           results_rank$MAPE_rank)
)

ggplot(ranking_long, aes(x = Metric, y = Model, fill = Rank)) +
  geom_tile() +
  geom_text(aes(label = Rank), size = 5) +
  labs(
    title = "Model Ranking by Forecast Metric (1 = Best)",
    x = "Metric",
    y = "Model"
  ) +
  theme_minimal()
```




```r
forecast_df <- data.frame(
  Time   = 1:length(test_ts),
  Actual = as.numeric(test_ts),
  Naive  = as.numeric(baseline_fit$mean),
  ARIMA  = as.numeric(arima_fc$mean),
  GARCH  = as.numeric(garch_fc_ts),
  ETS    = as.numeric(ets_fc$mean),
  NNAR   = as.numeric(nn_fc$mean)
)

forecast_long <- data.frame(
  Time = rep(forecast_df$Time, 6),
  Series = c(
    rep("Actual", nrow(forecast_df)),
    rep("Naive",  nrow(forecast_df)),
    rep("ARIMA",  nrow(forecast_df)),
    rep("GARCH",  nrow(forecast_df)),
    rep("ETS",    nrow(forecast_df)),
    rep("NNAR",   nrow(forecast_df))
  ),
  Value = c(
    forecast_df$Actual,
    forecast_df$Naive,
    forecast_df$ARIMA,
    forecast_df$GARCH,
    forecast_df$ETS,
```

```r
    forecast_df$NNAR
  )
)

ggplot(forecast_long, aes(x = Time, y = Value, color = Series)) +
  geom_line() +
  labs(
    title = "Actual vs Forecasted Volatility on the Test Set",
    x = "Time Index",
    y = "Volatility"
  ) +
  theme_minimal()
```